Advanced Computer Architecture 1st Term, 2026-2027 Integrating Project

Real-Time Audio Noise Reduction

Group 3

Alcasid, Aizy
Bilanes, Kendrick
Corpuz, Gerald
Roa, Luis

I. Abstract

This project proposes a real-time audio noise reduction system accelerated through hardware parallelization. The project's main objective is to process audio streams efficiently with low latency while improving the noise suppression performance across audio channels. It explores two primary frequency-domain algorithms: Power-domain Spectral Subtraction, which estimates and subtracts the noise spectrum from noisy signals, and the Wiener filter, which minimizes signal error using a signal-to-noise ratio estimate. Implemented in C using x86 AVX2 SIMD vectorization, the execution pipeline optimizes windowing, Fast Fourier Transforms (FFT), gain calculations, inverse FFT, and overlap-add reconstruction. Two parallelization strategies are evaluated: Version A vectorizes operations inside a single 512-sample frame to maintain low real-time latency, while Version B processes eight separate frames simultaneously across SIMD lanes to maximize total throughput. System performance will be tested based on execution time per frame, SNR improvement, and stream capacity, alongside optional CUDA GPU implementation comparisons.

Index Terms: Assembly Language, x86 AVX2 SIMD, Fast Fourier Transforms, Power-domain Spectral Subtraction, Wiener Filter

II. Description

Real-time audio noise reduction is essential for improving speech clarity in digital audio systems. Standard noise reduction pipeline cuts 16 kHz mono audio into short, overlapping 512-sample frames and uses the Fast Fourier Transform (FFT) to convert time-domain samples into frequency bins. This project has two options for classic algorithms to clean noisy audio. The Power-domain Spectral Subtraction, which estimates stationary background noise during initial silent frames and subtracts its power from the signal, and the Wiener Filter, which minimizes signal error by adjusting bin gains based on signal-to-noise ratio (SNR) estimates. Spectral Subtraction follows the foundational noise suppression techniques introduced by Boll (1979) and Berouti et al. (1979), whereas the decision-directed Wiener filter incorporates the a priori SNR estimation proposed by Scalart & Vieira Filho (1996) and Ephraim & Malah (1984).

To achieve real-time performance without processing delays, the algorithms are written in C and x86-64 assembly using NASM, compiled within Visual Studio 2022, and accelerated via AVX2 SIMD instructions. The system evaluates two parallelization approaches: Version A vectorizes operations within a single audio frame to maintain minimal real-time latency, while Version B processes eight separate audio frames simultaneously across SIMD lanes to maximize total channel throughput. System success is evaluated by comparing execution time per frame, signal-to-noise ratio improvement, and real-time stream capacity across sequential CPU, SIMD-accelerated CPU, and optional CUDA GPU implementations.

III. Proposed Algorithm

A. Power-domain Spectral Subtraction (Option 1)

To reduce noise in digital audio, one of the most popular methods is spectral subtraction: estimate the noise spectrum, then subtract it from the spectrum of the noisy signal.
First, the audio is cut into short, overlapping frames, and the Fast Fourier Transform (FFT) converts each frame from time samples into frequency bins. Each bin is a complex number with a real part and an imaginary part, which together hold its strength (magnitude) and timing (phase). Next, the power of each bin is computed by adding the square of the real part and the square of the imaginary part. Power is used because the noise's complex value changes randomly from frame to frame, so it cannot be predicted, but its average power stays roughly the same. Also, the powers of speech and noise add together on average, because they come from independent sources.

The noise power is estimated by averaging the power of the first K frames, which are assumed to contain only noise. For every frame, this noise power is subtracted from the frame's power. A factor $\alpha > 1$ subtracts a little extra, and a floor β keeps the result from dropping to zero or below, which avoids odd "musical" sounds. The cleaned power is turned into a gain between 0 and 1. The gain is multiplied onto both the real and imaginary parts, so the loudness of each bin is corrected while the original phase is kept. Finally, an inverse FFT converts each frame back to time samples, and the frames are overlap-added to rebuild the noise-reduced audio.

***Pseudocode* (original unparallelized pipeline):**

```
Input:  x[0..L-1]    noisy audio, 16 kHz mono
Params: N = 512, hop = 256, K, α, β, ε
Output: out[0..L-1]  noise-reduced audio

// Setup
for n = 0..N-1:
    w[n] = sqrt( 0.5 * (1 - cos(2π·n / N)) )     // sqrt-Hann
window
num_frames = floor((L - N) / hop) + 1

// Step 1: estimate the noise power from the first K frames
for k = 0..N-1:
    Nhat[k] = 0

for t = 0..K-1:
    for n = 0..N-1:
        frame[n] = w[n] * x[t*hop + n]           // window
    (re, im) = FFT(frame)                        // time ->
frequency
    for k = 0..N-1:
        Nhat[k] += re[k]*re[k] + im[k]*im[k]     // power

for k = 0..N-1:
```

```
    Nhat[k] = Nhat[k] / K                          // average

// Step 2: process every frame
for k = 0..L-1:
    out[k] = 0

for t = 0..num_frames-1:
    for n = 0..N-1:
        frame[n] = w[n] * x[t*hop + n]             // window

    (re, im) = FFT(frame)                          // forward FFT

    for k = 0..N-1:
        P = re[k]*re[k] + im[k]*im[k]              // power
spectrum
        G = sqrt( max(P - α*Nhat[k], β*Nhat[k]) / (P + ε) )   //
gain
        re[k] = G * re[k]                          // apply gain
        im[k] = G * im[k]

    y = IFFT(re, im)                               // back to time
samples

    for n = 0..N-1:
        out[t*hop + n] += w[n] * y[n]              // synthesis
window + overlap-add
```

B. Wiener Filter (Option 2)

The Wiener filter is another frequency-domain method that uses the same frame structure as spectral subtraction: window, FFT, power spectrum, noise estimate, then a per-bin gain, inverse FFT, and overlap-add. Instead of subtracting the noise power, it computes the gain that minimizes the mean-square error between the estimated and the clean speech. For each bin, the gain is $G[k] = \frac{\xi[k]}{(1+\xi[k])}$. Where $\xi[k]$ is a prior signal-to-noise ratio (SNR), for this case, it is the ratio between the power of the speech and the power of the noise in that bin. Bins dominated by speech have a larger ξ and a gain near 1. Bins dominated by noise have a small ξ and a gain near 0. As in spectral subtraction, the gain is a real number applied to both the real and imaginary parts, so the noisy phase is kept. The true speech power is unknown, so ξ is estimated. Let $y[k] = \frac{P[k]}{N[k]}$ be the a posteriori SNR. An estimated value, the decision-directed estimate (Scalart & Vieira Filho, 1996) is to be used: $\xi[k] = \frac{a \cdot \hat{S}_{prev}[k]^2}{N[k]+(1-a)\cdot max(y[k]-1,\,0)}$, where $\hat{S}_{prev}[k]^2$ is the cleaned power of the previous frame and a is about 0.98. It is smoother and reduces musical noise, but each frame depends on the previous one.

***Pseudocode* (original unparalleled pipeline):**

Input: x[0..L-1] noisy audio, 16 kHz mono

```
Params: N = 512, hop = 256, K, a, ξmin, ε
Output: out[0..L-1]  noise-reduced audio

// Setup: window w[n] and the noise estimate Nhat[k] as in
Option 1
// (average power of the first K frames)

for k = 0..N-1:
    prevS2[k] = 0                                  //
decision-directed only

for t = 0..num_frames-1:
    for n = 0..N-1:
        frame[n] = w[n] * x[t*hop + n]             // window
    (re, im) = FFT(frame)                          // forward FFT

    for k = 0..N-1:
        P     = re[k]*re[k] + im[k]*im[k]          // power
spectrum
        gamma = P / (Nhat[k] + ε)                  // a posteriori
SNR

        // decision-directed:
        xi = a * prevS2[k] / (Nhat[k] + ε) + (1 - a) *
max(gamma - 1, 0) // xi = max(xi, ξmin)
        G = xi / (1 + xi)                          // Wiener gain
        re[k] = G * re[k]                          // apply gain
        im[k] = G * im[k]
        prevS2[k] = G * G * P                      // save for
next frame

    y = IFFT(re, im)
    for n = 0..N-1:
        out[t*hop + n] += w[n] * y[n]              // synthesis
window + overlap-add
```

IV. Parallelization Approach

For Power-Domain Spectral Subtraction

The audio is split into numerous, overlapping frames (via window function, FFT, noise estimation, spectral subtraction, inverse FFT, and overlap-add in that order). The system then processes audio in the overlapping frames. The parameters are 16kHz mono, float32, N = 512 (32

ms frames), 50% overlap (hop 256 = 16 ms), and sqrt Hann windows for analysis and synthesis. In Boll's method of spectral subtraction (1979), stationary noise is suppressed by subtracting the noise spectrum measured during non-speech activity. Berouti et al. (1979) further improved this method with a subtraction factor of $\alpha > 1$ and a spectral floor of $0 < \beta < 1$ to control residual noise. The gain changes the magnitude and the phase is taken from the noisy signal. Every stage aside from the FFT stage involves element-wise operations across frequency bins, which is ideal for SIMD. FFT is the only part that requires restructuring.

Stage	Parallel Work	AVX2 Approach (8 floats per register)
Int16 to float, window	Per sample (one sample)	vpmovsxwd, vcvtdq2ps, vmulps
Forward FFT	Butterflies (one pair-wise add, subtract, or multiply inside FFT)	*See FFT Vectorization below
Power spectrum	Per bin (frequency bin)	vmulps + vfmadd231ps on split re and im
Noise profile	Per bin	vaddps, then one vmulps by 1/K
Gain $G = \sqrt{(\max(P - \alpha N, \beta N)/P)}$	Per bin	vfnmadd/vsubps, vmaxps, vdivps, vsqrtps
Apply gain	Per bin	vmulps on re and im
Inverse FFT	Butterflies	Swap re/im pointers, scale by 1/N
Synthesis window + overlap-add	Per sample	vmulps, vaddps at the hop offset
Float to int16	Per sample	vcvtps2dq, vpackssdw, vpermq

**note: K = # of frames, N = # of samples in each frame, re = real part, im = imaginary part, P = power spectrum (P[k] = re[k]^2 + im[k]^2)*

FFT Vectorization

Version A: vectorize inside one frame (keep real-time latency)

- At startup
    1. Allocate 32-byte-aligned re[] and im[] arrays (512 floats each).
    2. Precompute the twiddle tables, laid out in the order each stage reads them.
    3. Build the noise profile Nhat[k] from the first K frames. Store it, and the gain array, in bit-reversed bin order.
- For each frame

1. Convert int16 to float, multiply by the window, and write the result to re[]. Set im[] to 0.
2. Forward DIF, stages 1 – 6 (spans 256, 128, 64, 32, 16, 8).
    - Has pure vector code with contiguous loads, 8 butterflies per loop iteration, and no shuffles.
3. Forward DIF, stages 7 – 9 (spans 4, 2, 1).
    - These use the in-register kernel with shuffles (vunpcklps, vshufps, vpermps), applied to each group of 8 values.
4. Per-bin steps on the bit-reversed spectrum: power, subtract, gain, apply gain.
    - This is 64 loop iterations of 8 bins.
5. Swap the re/im pointers (free), then run the inverse DIT, spans 1, 2, 4 first (in-register kernel).
6. Inverse DIT, spans 8 to 256 (pure vector).
7. Swap the pointers back and scale by 1/N. The output is now in natural order.
8. Multiply by the synthesis window, overlap-add, and convert to int16.

Version B: one frame per SIMD lane

- At startup
    1. Allocate aligned arrays of 512 × 8 floats each for re and im. Element k of lane f lives at index = k*8 + f.
    2. Precompute scalar twiddles. Each butterfly uses one twiddle for all lanes.
    3. Build the noise profile from the first K frames (for static profile only).
- For every 8 frames
    1. Interleave the 8 frames into the lane layout.
        - This is an 8×8 transpose and is the only shuffle-heavy step. If 8 interleaved audio channels are processed, the data is already in this layout, so this step is skipped.
    2. Window: multiply each vector by w[n] broadcast to all lanes (vbroadcastss). Set im to 0.
    3. Forward DIF, all 9 stages.
        - Every stage has the same structure: load the vectors for elements a and b, broadcast the twiddle, complex multiply, then add and subtract.
        - There are no shuffles, and “span” only changes which vectors are paired.
    4. Per-bin steps: power, subtract, gain, apply gain. Nhat[k] is broadcast to all lanes, or loaded as a vector if each channel has its own profile.
    5. Swap the re/im pointers, then run the inverse DIT, all 9 stages (spans 1 to 256).
    6. Swap back and scale by 1/N.
    7. Multiply by the synthesis window.
    8. Overlap-add and convert to int16 (for 8 channels, this is a vertical add with the previous hop in the same lane).
        - For 8 consecutive frames of one stream, transpose back to normal layout first, because neighboring frames sit in different lanes.

Arrays are aligned to 32 bytes (64 for AVX-512) so aligned loads (vmovaps) can be used. In Version A, the re and im arrays for one 512 - point frame take about 4 KB, and with the

twiddle, window, and noise tables, the working set is roughly 12 KB. This fits in an L1 data cache, so the pipeline is compute-bound. In Version B, 8 frames are held at once, so re and im take 32 KB (about 38 KB with tables), which is near the size of a typical data cache. Some cache traffic is therefore expected. If Version A later adopts real-FFT packing, the 257 bins are padded to a multiple of 8 or handled with masked operations (vmaskmovps, or AVX-512 mask registers). Version B needs no bin padding, but its last batch is padded with silent frames if the frame count is not a multiple of 8.

One 16 kHz stream is cheap since a 512 FFT would take microseconds against a 16ms budget. To make comparisons meaningful, reports about how many simultaneous streams each version sustains in real time will be made. On the GPU side, CUDA would use one thread per bin for the fused elementwise stages and a batched FFT (cuFFT or a one-block-per-frame kernel). PCIe transfer and launch overhead usually makes a GPU lose on single small frames, so it is only effective with many frames or channels batched together.

For Wiener Filter

The Wiener filter uses the same frame structure as the spectral subtraction: window function, FFT, noise estimation, spectral subtraction, inverse FFT, and overlap-add in that order. The parameters also remain the same: 16kHz mono, float32, N = 512 (32 ms frames), 50% overlap (hop 256 = 16 ms), and sqrt Hann windows for analysis and synthesis. Though the gain stage differs. Instead of subtracting the noise spectrum, the filter multiplies each frequency bin by a gain of G[k] = ξ[k] / 1+ ξ [k], where ξ is the ratio of speech power to noise power in a bin. Bins dominated by speech have a gain nearing 1, while bins dominated by noise have a gain nearing 0, compared to spectral subtraction, where the gain changes the magnitude and the phase is taken from a noisy signal.

ξ has two ways of being estimated. One of them uses only the current frame: ξ = max(y - 1, ξ min), where y = P / (Nhat) is the ‘a posteriori SNR.’ The decision-directed version (Scalart & Vieira Filho, 1996) blends in the previous frame’s cleaned output: ξ = a*|Ŝprev|^2 / Nhat + (1 − a)*max(γ − 1, 0). This formula proves to be smoother but requires the previous frame’s result.

Every stage aside from the FFT stage involves element-wise operations across frequency bins, so the SIMD approach is the same as for spectral subtraction, though the gain stage needs one division per bin and no square root, thus vsqrtps is no longer used.

Stage	Parallel Work	AVX2 Approach (8 floats per register)
Int16 to float, window	Per sample (one sample)	vpmovsxwd, vcvtdq2ps, vmulps
Forward FFT	Butterflies (one pair-wise add, subtract, or multiply inside FFT)	*See FFT Vectorization below
Power spectrum	Per bin (frequency bin)	vmulps + vfmadd231ps on split re and im

Noise profile	Per bin	vaddps, then one vmulps by 1/K ; invN[k] = 1/Nhat[k] computed once at startup via vdivps
A posteriori SNR y = P * invN	Per bin	vmulps
A posteriori SNR (simple) y = max(y - 1, ξ min)	Per bin	vsubps, vmaxps, vmulps, vfmadd231ps (uses the prevS2 array)
Gain G = ξ / (1 + ξ)	Per bin	Vaddps, vdivps
Apply gain	Per bin	vmulps on re and im
Save prevS2 = G^2 * P	Per bin	vmulps
Inverse FFT	Butterflies	Swap re/im pointers, scale by 1/N
Synthesis window + overlap-add	Per sample	vmulps, vaddps at the hop offset
Float to int16	Per sample	Vcvtps2dq, vpackssdw, vpermq

**note: a = smoothing weight (about 0.98 as a starting value)*

FFT Vectorization

Version A and Version B remain the same as the spectral subtraction algorithm with only a change in the increment of bins from “power, subtract, gain, apply gain” to “power, γ, ξ, gain, apply gain, prevS2 (for decision-directed version).”

Version A: vectorize inside one frame

The simple Wiener filter fits in perfectly without having to change anything. With the decision-directed version, a prevS2 array of 512 floats (~12 KB) is added and stored in the same bit-reversed bin order as Nhat. This brings the set to ~14 KB, which still fits in L1 data cache.

Version B: one frame per SIMD lane

The simple Wiener filter can fit perfectly again, but the decision-directed version cannot be used when 8 lines hold 8 consecutive frames of one stream since each frame needs the result of the frame before. It only works when the lines are independent channels with one prevS2 value per bin per lane, resulting in an array of 512x8 floats (16 KB), which increases the working set to ~54 KB, which is above a typical L1 data cache, so more L2 traffic is expected

Since FFT and inverse FFT dominate the work, the choice of gain function is expected to change total speed only a little. This will be measured by running the same noisy files through methods and comparing SNR improvement, time per frame, and the number of real-time streams happening at the same time.

V. Implementation

The project will be written in C and x86-64 assembly using Visual Studio Community 2022. The C code handles the main program flow: reading and writing WAV files, splitting the audio into frames, running the sequential baseline, timing, and computing SNR. The AVX2 kernels are written in NASM (x64) in separate .asm files and called from the C code. Each .asm file is added to the project and set to Item Type "Custom Build Tool", with the command line "nasm -f win64 kernels.asm" and the output "kernels.obj", so the object file is linked automatically. The target machine is a laptop or desktop with an x86-64 CPU that supports AVX2 and FMA, since the kernels use instructions such as "vmulps" and "vfmadd231ps". As an optional extension, we may add a CUDA version using cuFFT on an NVIDIA GPU for an extra comparison.

VI. Mid-project Milestones

Week	Milestone	Deliverable
7	Sequential baseline	Visual Studio 2022 project set up with NASM (Custom Build Tool) and a test call from C to assembly. Scalar C program for spectral subtraction and Wiener filter (WAV I/O, window, FFT/IFFT, gain, overlap-add). Test files with SNR measurement. Baseline timing.
8	SIMD kernels	AVX2 kernels in NASM for int16 to float, window, power spectrum, gain (both methods), apply gain, overlap-add, float to int16. Each tested against the scalar version.
9	FFT vectorization and integration	Vectorized FFT/IFFT for Version A integrated into the full pipeline. Output matched the baseline
10	Mid-project demo	Working baseline plus Version A, with first timing results (time per frame, speedup, SNR improvement)
11	Version B	One-frame-per-lane version with the 8x8 transpose, tested

		on 8 streams.
12	Benchmarking and optional CUDA	Full comparison of baseline, A and B for both methods (time per frame, real-time streams, SNR improvement).
13	Final submission and demo	Github repository, sequential vs. parallel execution time comparison and optional conference paper.

VII. References

Boll, S. F. (1979). *Suppression of Acoustic Noise in Speech Using Spectral Subtraction* (IEEE TRANSACTIONS ON ACOUSTICS, SPEECH, AND SIGNAL PROCESSING ed., VOL. ASSP-27, NO. 2,).

Franchetti, F., & Puschel, M. (2003). *Short Vector Code Generation for the Discrete Fourier Transform,* International Parallel and Distributed Processing Symposium: Proceedings: April 22-26, 2003, Nice, France. IEEE Computer Society.

Franchetti, F., & Puschel, M. (2008). *Generating SIMD Vectorized Permutations*. Electrical and Computer Engineering, Carnegie Mellon University.

Frigo, M., & Johnson, S. G. (2005). *The Design and Implementation of FFTW3* (Vol. 93, no. 2). Proc. IEEE.

Lee, V. W., Kim, C., Chhugani, J., Deisher, M., Kim, D., Nguyen, A. D., Satish, N., Smelyanskiy, M., Chennupaty, S., Hammarlund, P., Singhal, R., & Dubey, P. (2010). *Debunking the 100x GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU*. https://www.researchgate.net/publication/234783051_Debunking_the_100x_GPU_vs_CPU_myth_an_evaluation_of_throughput_computing_on_CPU_and_GPU

Schwartz, R., Berouti, M., & Makhoul, J. (1979, May). *(PDF) Enhancement of speech corrupted by acoustic noise*. ResearchGate. Retrieved October 18, 2026, from https://www.researchgate.net/publication/224738211_Enhancement_of_speech_corrupted_by_acoustic_noise

Ephraim, Y., & Malah, D. (1984). Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator. *IEEE Transactions on Acoustics, Speech, and Signal Processing, 32*(6), 1109–1121.

Scalart, P., & Vieira Filho, J. (1996). Speech enhancement based on a priori signal to noise estimation. *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2, 629–632.

Lim, J. S., & Oppenheim, A. V. (1979). Enhancement and bandwidth compression of noisy speech. *Proceedings of the IEEE, 67*(12), 1586–1604.

Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation, 19*(90), 297–301.

Kothapalli, K., Banerjee, D. S., Narayanan, P. J., et al. (2013). CPU and/or GPU: Revisiting the GPU vs. CPU myth. arXiv:1303.2171.

https://sistc.com/spectral-subtraction-audio-noise-reduction/